Data Structures

BS (CS) _Fall_2025

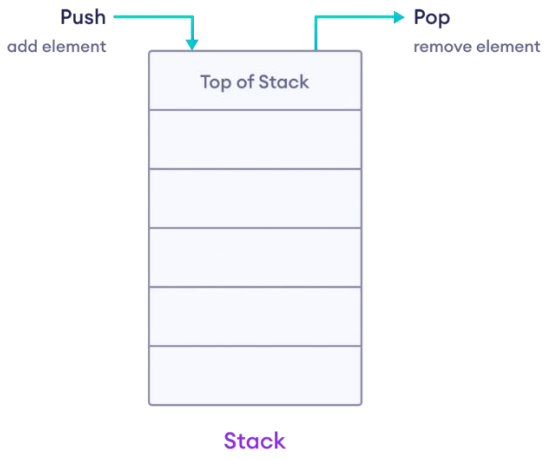
# Lab_08 Manual


## Learning Objectives:

1. Stacks and Queues

# Stacks

The stack data structure follows the **LIFO (Last In First Out)** principle. That is, the element added last will be removed first.




## Stack Operations

In C++, the stack can perform different operations including.

- push() adds an element into the stack
- pop() removes an element from the stack
- top() returns the element at the top of the stack
- size() returns the number of elements in the stack
- empty() returns true if the stack is empty

## Stacks Declaration using Linked List

A stack can be implemented using a **linked list** where we maintain:

A **Node structure/class** that contains:

- data → to store the element.
- next → pointer/reference to the next node in the stack.

A pointer/reference top that always points to the current **top node** of the stack.

- Initially, top = null to represent an empty stack.

## Push:

Insertion always happens from the top

```
    void push(int x) {
        Node* temp = new Node(x);
        temp->next = top;
        top = temp;

        count++;
    }
```

## Pop:

Removal always happens from the top as well.

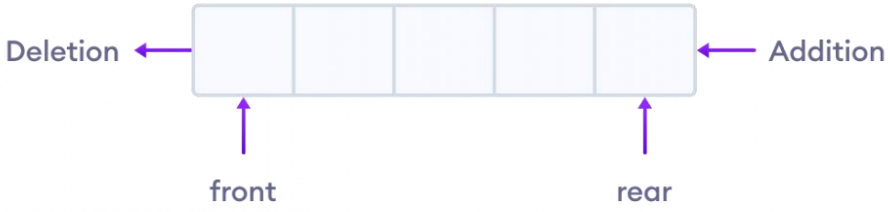
```
    int pop() {
        if (top == NULL) {
            cout << "Stack Underflow" << endl;
            return -1;
        }
        Node* temp = top;
        top = top->next;
        int val = temp->data;

        count--;
        delete temp;
        return val;
    }
```

## Time Complexities:

| Operation | Time Complexity | Description |
|---|---|---|
| • Push | O(1) | Adding an element to the top of the stack. |
| • Pop | O(1) | Removing the top element from the stack. |
| • Peek / Top | O(1) | Viewing the top element without removing it. |
| • Is Empty | O(1) | Checking if the stack has no elements. |
| • Search | O(n) | Finding the position of an element in the stack. |

# Queues

The queue data structure follows the **FIFO (First In First Out)** principle where elements that are added first will be removed first.




## Queue Operations

In C++, a queue can perform different operations including.

- enqueue() Inserts an element at the back of the queue
- dequeue() Removes an element from the front of the queue
- front() Returns the first element of the queue
- back() Returns the last element of the queue
- size() Returns the number of elements in the queue
- empty() Returns true if the queue is empty.

## Queue Declaration using Linked List

To implement a queue with a linked list, we maintain:

A **Node structure/class** that contains:

- data → to store the element
- next → pointer/reference to the next node in the queue

**Two pointers/references**:

- front → points to the first node (head of the queue)
- rear → points to the last node (tail of the queue)

## Enqueue:

```
void enqueue(int new_data) {
    Node* node = new Node(new_data);
    if (isEmpty()) {
        front = rear = node;
    } else {
        rear->next = node;
        rear = node;
    }
}
```

## Dequeue:

```
int dequeue() {
       if (isEmpty()) {
           cout << "Queue Underflow" << endl;
           return -1;
       }

       Node* temp = front;
       int removedData = temp->data;
       front= front->next;

       if (front == nullptr) rear = nullptr;
       delete temp;
       return removedData;
}
```

## Time Complexities:

| Operation | Time Complexity | Description |
|---|---|---|
| • Enqueue (Add) | O(1) | Adding an element to the back/rear of the queue. |
| • Dequeue (Remove) | O(1) | Removing an element from the front of the queue. |
| • Front / Peek | O(1) | Viewing the front element without removing it. |
| • Is Empty | O(1) | Checking if the queue has no elements. |
| • Search | O(n) | Finding the position of an element in the queue. |